AI COURSE – II AIML/REC

CATEGORY 4 – PYTHON PROGRAMS ONLY

1. Breadth First Search (BFS)

```
from collections import deque

def bfs(graph, start, goal):
    queue = deque([[start]])
    visited = set()

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            return path

        if node not in visited:
            visited.add(node)

            for neighbour in graph[node]:
                if neighbour not in visited:
                    queue.append(path + [neighbour])

    return None


graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["G"],
    "F": [],
    "G": []
}

start = "A"
goal = "G"

path = bfs(graph, start, goal)
print("BFS Path:", path)
```

2. Depth First Search (DFS)

```
def dfs(graph, start, goal):
    stack = [(start, [start])]
    visited = set()

    while stack:
        node, path = stack.pop()

        if node == goal:
            return path

        if node not in visited:
            visited.add(node)

            for neighbour in reversed(graph[node]):
                if neighbour not in visited:
                    stack.append((neighbour, path + [neighbour]))

    return None


graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["G"],
    "F": [],
    "G": []
}
```

```
start = "A"
goal = "G"

path = dfs(graph, start, goal)
print("DFS Path:", path)
```

3. Depth Limited Search (DLS)

```
def depth_limited_search(graph, node, goal, limit, path=None):
    if path is None:
        path = [node]

    if node == goal:
        return path

    if limit == 0:
        return None

    for neighbour in graph[node]:
        if neighbour not in path:
            result = depth_limited_search(
                graph, neighbour, goal, limit - 1, path + [neighbour]
            )

            if result is not None:
                return result

    return None


graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["G"],
    "F": [],
    "G": []
}

start = "A"
goal = "G"
depth_limit = 3

path = depth_limited_search(graph, start, goal, depth_limit)

if path:
    print("DLS Path:", path)
else:
    print("Goal not found within depth limit")
```

4. Bidirectional Search

```
from collections import deque

def bidirectional_search(graph, start, goal):
    if start == goal:
        return [start]

    forward_queue = deque([start])
    backward_queue = deque([goal])

    forward_parent = {start: None}
    backward_parent = {goal: None}

    while forward_queue and backward_queue:

        # Expand from the start node.
        current = forward_queue.popleft()

        for neighbour in graph[current]:
            if neighbour not in forward_parent:
                forward_parent[neighbour] = current
                forward_queue.append(neighbour)

            if neighbour in backward_parent:
                meeting = neighbour
                return build_path(
                    meeting, forward_parent, backward_parent
                )

        # Expand from the goal node using the reverse graph.
        current = backward_queue.popleft()

        for neighbour in reverse_graph[current]:
            if neighbour not in backward_parent:
```

```
                backward_parent[neighbour] = current
                backward_queue.append(neighbour)

            if neighbour in forward_parent:
                meeting = neighbour
                return build_path(
                    meeting, forward_parent, backward_parent
                )

    return None


def build_path(meeting, forward_parent, backward_parent):
    path = []
    node = meeting

    while node is not None:
        path.append(node)
        node = forward_parent[node]

    path.reverse()
    node = backward_parent[meeting]

    while node is not None:
        path.append(node)
        node = backward_parent[node]

    return path


graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["G"],
    "F": [],
    "G": []
}

# Reverse edges for searching backward from the goal.
reverse_graph = {node: [] for node in graph}

for node in graph:
    for neighbour in graph[node]:
        reverse_graph[neighbour].append(node)

start = "A"
goal = "G"

path = bidirectional_search(graph, start, goal)
print("Bidirectional Search Path:", path)
```